Systems of Linear Equations

Systems of Linear Equations

Direct Methods

- Gaussian Elimination
- LU Factorization
- Cholesky Factorization

Iterative Methods

- Jacobi Method
- Gauss–Seidel Method

Systems of Linear Equations

What is a system of linear equations?

A *system of linear equations* is a collection of linear equations

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned}$$

We write this compactly as $Ax = b$

Systems of Linear Equations

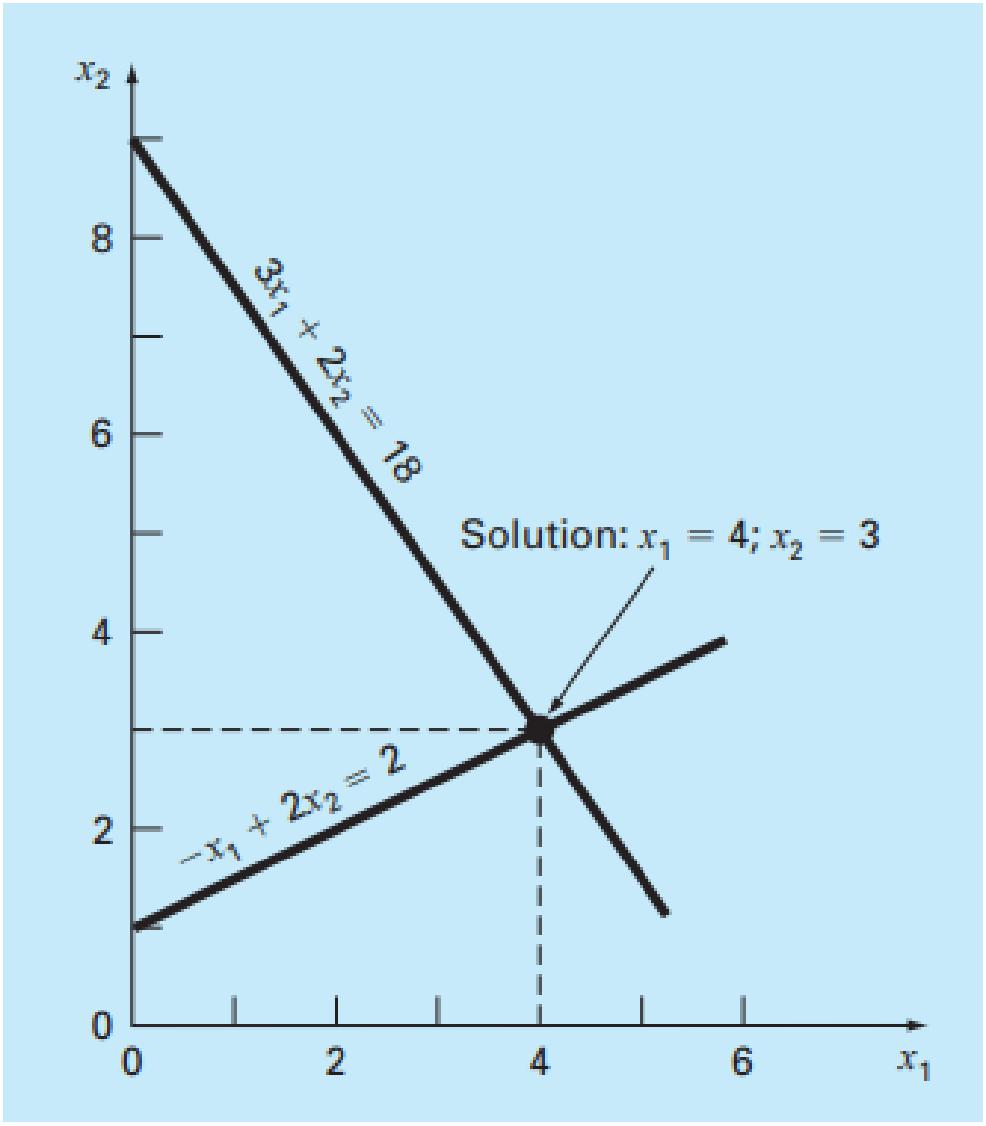
We write this compactly as $Ax = b$, where

- A is the coefficient matrix (size $n \times n$)
- x is the unknown vector $(x_1, \ldots, x_n)^T$
- b is the right-hand side vector $(b_1, \ldots, b_n)^T$

Our Target: find x satisfying $Ax = b$.

Graphical representation for two equations

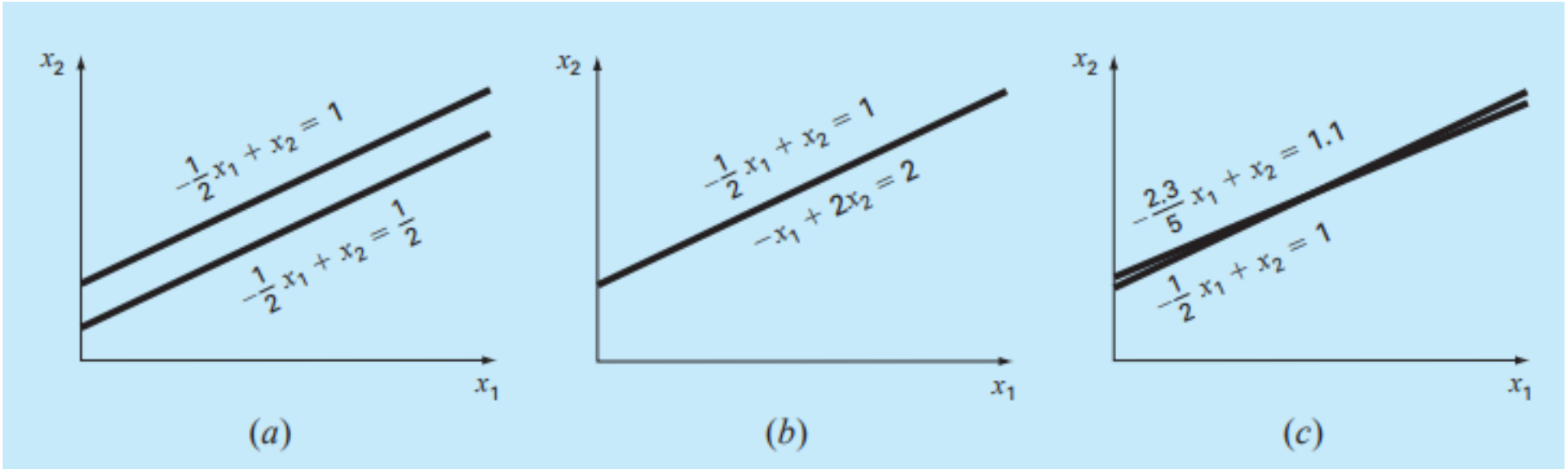
$$3x_1 + 2x_2 = 18$$
$$-x_1 + 2x_2 = 2$$



Graphical representation for two equations

Three possible geometric cases

- one intersection → unique solution
- parallel (no intersection) → no solution
- coincident (infinitely many intersections) → infinite solutions



Augmented matrix representation

$$
\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\
a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\
&\vdots \\
a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n
\end{aligned}
$$



$A = [a_{ij}]$, $x = (x_1, \ldots, x_n)^T$, $b = (b_1, \ldots, b_n)^T$. Then $Ax = b$.

Augmented matrix representation

$$
\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\
a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\
&\vdots \\
a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n
\end{aligned}
$$

$A = [a_{ij}]$, $x = (x_1, \ldots, x_n)^T$, $b = (b_1, \ldots, b_n)^T$. Then $Ax = b$.

$$
(A|b) = \left(\begin{array}{ccc|c}
a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\
\vdots & & & \vdots & \vdots \\
a_{n1} & \cdots & & a_{nn} & b_n
\end{array}\right)
$$

Elementary row operations

Row swap: $R_i \leftrightarrow R_j$

Row scale: multiply a row by nonzero scalar λ: $R_i \leftarrow \lambda R_i$

Row replacement: add a multiple of one row to another: $R_i \leftarrow R_i + \lambda R_j$

Gaussian elimination

1. We will convert augmented matrix to an *upper triangular* form (zeros below main diagonal).

2. Then solve by backward substitution.

Gaussian elimination

Process => two phases:

1. **Forward elimination:** make all entries below the diagonal zero, column by column.

2. **Backward substitution:** solve variables starting from last row up.

*we use **multiplier** m_{ik} to eliminate entries in column k below row i.

Gaussian elimination - Example

$$x_1 + x_2 + 0x_3 = 2$$
$$2x_1 + x_2 - x_3 = 2$$
$$3x_1 - x_2 - x_3 = 1$$

Augmented matrix $(A|b)$:

$$\left(\begin{array}{ccc|c} 1 & 1 & 0 & 2 \\ 2 & 1 & -1 & 2 \\ 3 & -1 & -1 & 1 \end{array}\right)$$

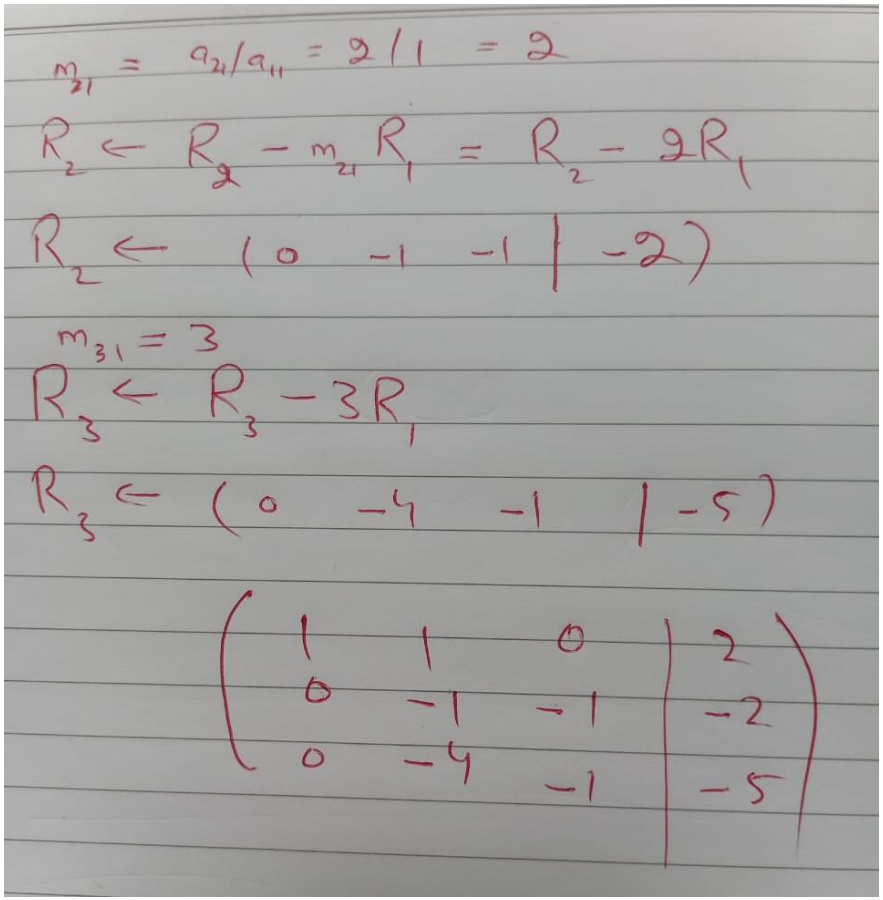
Gaussian elimination - Example

$$\left(\begin{array}{ccc|c} 1 & 1 & 0 & 2 \\ 2 & 1 & -1 & 2 \\ 3 & -1 & -1 & 1 \end{array}\right)$$

$$\left(\begin{array}{ccc|c} 1 & 1 & 0 & 2 \\ 0 & -1 & -1 & -2 \\ 0 & -4 & -1 & -5 \end{array}\right)$$

Update $R_3 \leftarrow R_3 - 4R_2$

$$\left(\begin{array}{ccc|c} 1 & 1 & 0 & 2 \\ 0 & -1 & -1 & -2 \\ 0 & 0 & 3 & 3 \end{array}\right)$$



Gaussian elimination - Example

$$\left(\begin{array}{ccc|c} 1 & 1 & 0 & 2 \\ 0 & -1 & -1 & -2 \\ 0 & 0 & 3 & 3 \end{array}\right)$$

Backward substitution

From last row: $3x_3 = 3 \Rightarrow x_3 = 3/3 = 1$.

Second row: $-1 \cdot x_2 + (-1) \cdot x_3 = -2$.

Plug $x_3 = 1$: $-x_2 - 1 = -2$ so $-x_2 = -1 \Rightarrow x_2 = 1$.

First row: $x_1 + x_2 = 2$. Plug $x_2 = 1$: $x_1 + 1 = 2 \Rightarrow x_1 = 1$.

Solution: $x = (1, 1, 1)^T$.

Gaussian elimination

We used multipliers:
$m_{21} = 2,\ m_{31} = 3, m_{32} = 4$

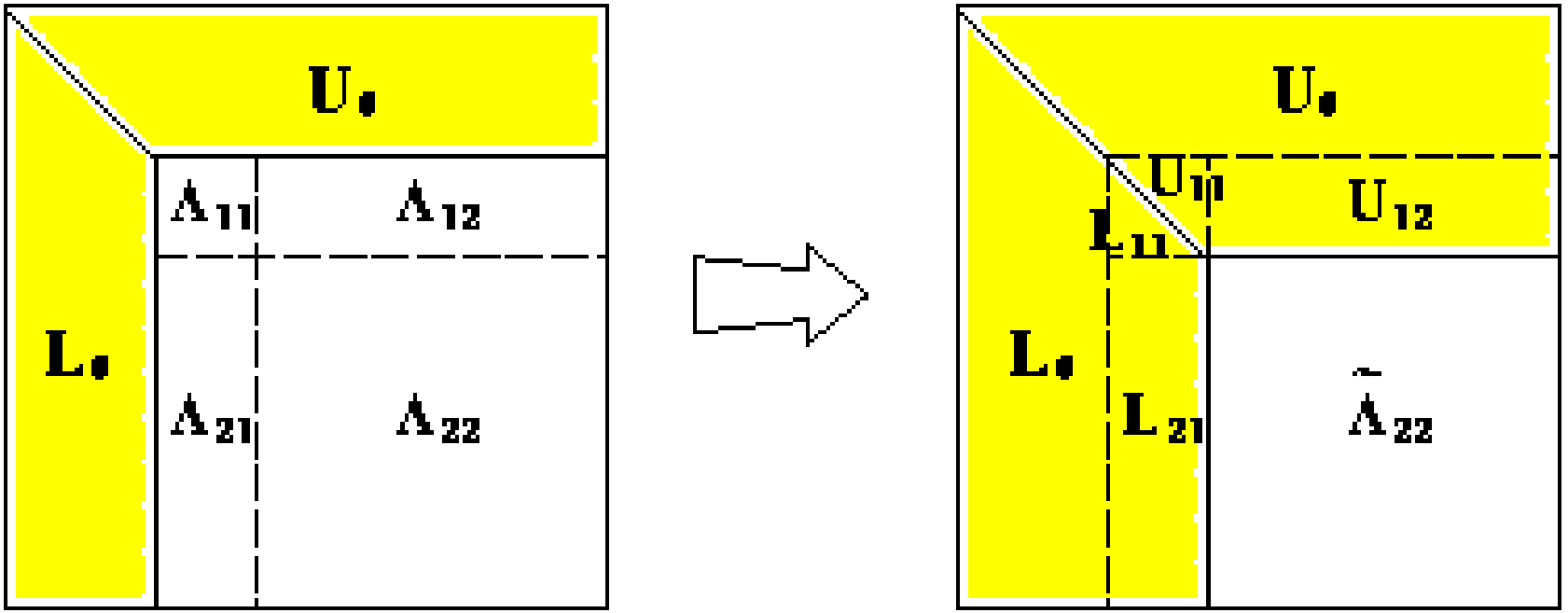
$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 4 & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 1 & 1 & 0 \\ 0 & -1 & -1 \\ 0 & 0 & 3 \end{pmatrix}$$

$$A = L \cdot U$$

LU factorization

Why LU?

If you need to solve $Ax = b$ for many different b but same A,
it's efficient to compute LU once (cost $O(n^3)$(
then use forward/back substitution for each b (cost $O(n^2)$)

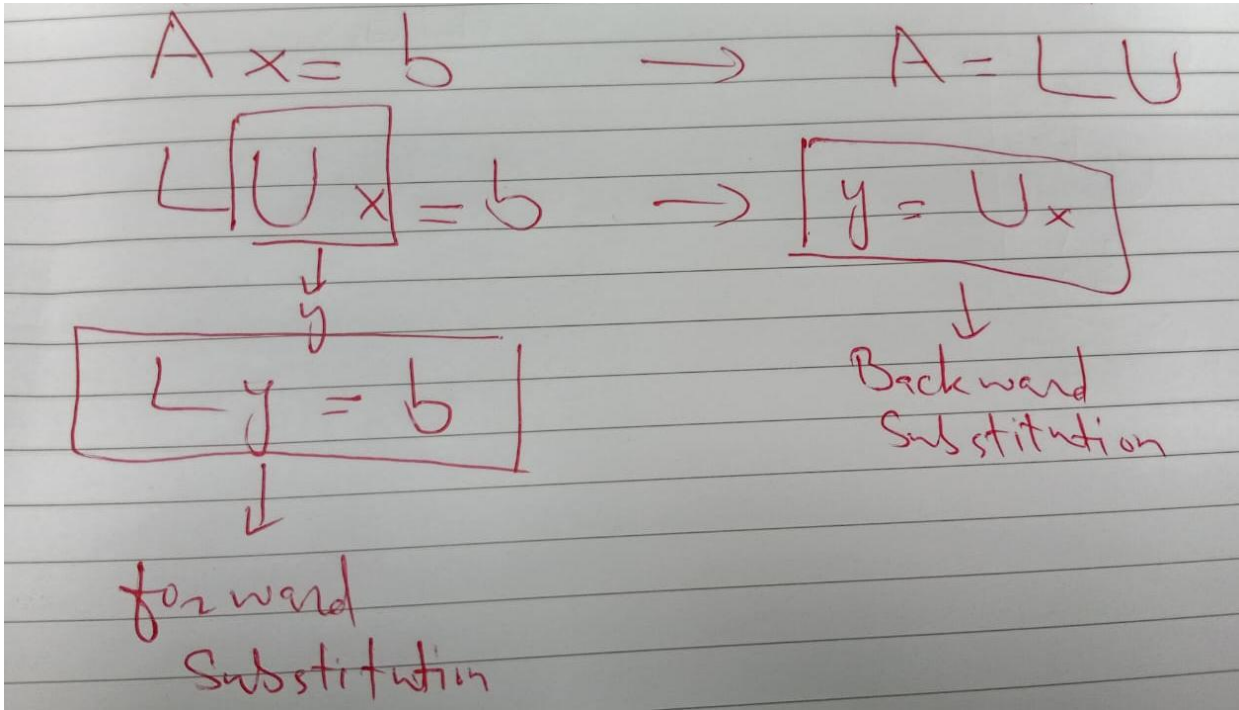


** **BLAS** Operations => Level 1, Level 2, Level 3

LU factorization

Solving with LU:

1. Compute $A = LU$.
2. Solve $Ly = b$ (forward substitution).
3. Solve $Ux = y$ (back substitution).



LU factorization - Pivoting

Problem without pivoting

If a pivot a_{kk} is zero (or very small), dividing by it is impossible (or numerically undefined).

Example:

$$A = \begin{pmatrix} 10^{-17} & 1 \\ 1 & 1 \end{pmatrix}, b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Exact solution is $x \approx (1, 1)$

But if we try to eliminate using row1 pivot 10^{-17} , we compute multiplier $m = \frac{1}{10^{-17}} = 10^{17}$

A huge number — rounding errors explode;
numerical answer can be garbage.

Solution: *Pivoting* —> swap rows so we use a safe pivot.

LU factorization - Pivoting

Partial pivoting

At step k, among rows $k..n$ pick the row with largest absolute value in column $k$and swap it into row k. No column swaps. This will improve numerical stability in practice.

Original:

$$A = \begin{pmatrix} 10^{-17} & 1 \\ 1 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Swap rows 1 and 2:

$$\begin{pmatrix} 1 & 1 \\ 10^{-17} & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Now pivot is 1 safe for operation

LU factorization with pivoting (PA = LU)

If we do row swaps during elimination, we get a *permutation matrix* P representing the swaps, and factorization becomes:

$$PA = LU$$

To solve $Ax = b$:

1. Compute $PA = LU$.
2. Compute Pb.
3. Solve $Ly = Pb$ (forward substitution).
4. Solve $Ux = y$.

The matrices L and U in the P A = LU factorization with pivoting are not the same as in the A = LU factorization without pivoting.

LU factorization - Pivoting

Scaled Partial Pivoting

- **Look at each row and find the biggest number in that row.**
- **When choosing the pivot row in a column**

  For each row, compare the number in the pivot column **divided by** the biggest number of that row. This gives a "scaled" value.
- **Pick the row with the biggest scaled value.**
- **Swap that row to the top (pivot position).**
- **Use that pivot row to eliminate (make zeros) below it.**
- **Move to the next column and repeat.**

Pivoting

Scaled Partial Pivoting

$$
\begin{aligned}
6x_1 + 2x_2 - x_3 &= 7 \\
x_1 + 0.1x_2 - 5x_3 &= -3.9 \\
2x_1 - x_2 + 7x_3 &= 9
\end{aligned}
$$

$$
\left[\begin{array}{cccc}
6 & 2 & -1 & 7 \\
1 & 0.1 & -5 & -3.9 \\
2 & -1 & 7 & 9
\end{array}\right]
$$

1. Look at each row and find the biggest number in that row.

R1 -> max(6,2,1) = 6
R2 -> max(1,0.1,5) = 5
R3 -> max(2,1,7) = 7

Scale factors (maximum absolute in each row): $s = [6, 5, 7]$.

`Row 1 will be the pivot row in this case`

Pivoting

Scaled Partial Pivoting

$$
\begin{aligned}
6x_1 + 2x_2 - x_3 &= 7 \\
x_1 + 0.1x_2 - 5x_3 &= -3.9 \\
2x_1 - x_2 + 7x_3 &= 9
\end{aligned}
$$

$$
\begin{bmatrix}
6 & 2 & -1 & 7 \\
1 & 0.1 & -5 & -3.9 \\
2 & -1 & 7 & 9
\end{bmatrix}
$$

$s = [6, 5, 7]$.

2. For each row, compare the number in the pivot column divided by the biggest number of that row. This gives a "scaled" value.

First column is the pivot column in this case

```
6 / 6 = 1
1 / 5 = 0.5
2 / 7 = 0.285
```

The biggest scaled value is 1 which is also from the row 1 so no swap required in this case

Pivoting

Scaled Partial Pivoting

$$
\begin{aligned}
6x_1 + 2x_2 - x_3 &= 7 \\
x_1 + 0.1x_2 - 5x_3 &= -3.9 \\
2x_1 - x_2 + 7x_3 &= 9
\end{aligned}
$$

$$
\begin{bmatrix}
6 & 2 & -1 & 7 \\
1 & 0.1 & -5 & -3.9 \\
2 & -1 & 7 & 9
\end{bmatrix}
$$

$s = [6, 5, 7]$.

3. Pick the row with the biggest scaled value.
4. Swap that row to the top (pivot position).
5. Use that pivot row to eliminate (make zeros) below it.

Pivoting

Scaled Partial Pivoting

$$
\begin{aligned}
6x_1 + 2x_2 - x_3 &= 7 \\
x_1 + 0.1x_2 - 5x_3 &= -3.9 \\
2x_1 - x_2 + 7x_3 &= 9
\end{aligned}
\qquad
\begin{bmatrix}
6 & 2 & -1 & 7 \\
1 & 0.1 & -5 & -3.9 \\
2 & -1 & 7 & 9
\end{bmatrix}
$$

$s = [6, 5, 7]$.

5. Use that pivot row to eliminate (make zeros) below it.

$$
\left[\begin{array}{ccc|c}
6 & 2 & -1 & 7 \\
0 & -0.233333333 & -4.833333333 & -5.066666667 \\
0 & -1.666666667 & 7.333333333 & 6.666666667
\end{array}\right]
$$

Again check the scale with the maximum value of that row, choose second column this time

Pivoting

Scaled Partial Pivoting

$$
\begin{aligned}
6x_1 + 2x_2 - x_3 &= 7 \\
x_1 + 0.1x_2 - 5x_3 &= -3.9 \\
2x_1 - x_2 + 7x_3 &= 9
\end{aligned}
$$

$$
\begin{bmatrix}
6 & 2 & -1 & 7 \\
1 & 0.1 & -5 & -3.9 \\
2 & -1 & 7 & 9
\end{bmatrix}
$$

$s = [6, 5, 7]$.

5. Use that pivot row to eliminate (make zeros) below it.

$$
\left[\begin{array}{ccc|c}
6 & 2 & -1 & 7 \\
0 & -1.666666667 & 7.333333333 & 6.666666667 \\
0 & -0.233333333 & -4.833333333 & -5.066666667
\end{array}\right]
$$

$$
\begin{bmatrix}
6 & 2 & -1 & 7 \\
0 & -1.666667 & 7.333333 & 6.666667 \\
0 & 0 & -5.86 & -6
\end{bmatrix}
$$

Pivoting

Full Pivoting

- Look at the whole remaining part of the matrix, not just one column.
- Find the single biggest number (in absolute value) in that whole part.
- Move that row up to the pivot row.
- Move that column left to the pivot column.
- Use that pivot row to eliminate (make zeros) below it.
- Go to the next smaller block of the matrix and repeat.
- After solving, put the variables back in the original order

Pivoting

Full Pivoting

$$\begin{aligned} 1x_1 + 1x_2 + 10x_3 &= 12 \\ 10x_1 + 1x_2 + 1x_3 &= 12 \\ 1x_1 + 10x_2 + 1x_3 &= 12 \end{aligned} \longrightarrow \left[\begin{array}{ccc|c} 1 & 1 & 10 & 12 \\ 10 & 1 & 1 & 12 \\ 1 & 10 & 1 & 12 \end{array}\right]$$

$$\max\left(\begin{bmatrix} 1 & 1 & \mathbf{10} \\ 10 & 1 & 1 \\ 1 & 10 & 1 \end{bmatrix}\right) = |10| = 10$$

Pivoting

Full Pivoting

$$
\begin{aligned}
1x_1 + 1x_2 + 10x_3 &= 12 \\
10x_1 + 1x_2 + 1x_3 &= 12 \\
1x_1 + 10x_2 + 1x_3 &= 12
\end{aligned}
\longrightarrow
\left[\begin{array}{ccc|c}
1 & 1 & 10 & 12 \\
10 & 1 & 1 & 12 \\
1 & 10 & 1 & 12
\end{array}\right]
$$

$$
\left[\begin{array}{ccc|c}
10 & 1 & 1 & 12 \\
1 & 1 & 10 & 12 \\
1 & 10 & 1 & 12
\end{array}\right]
$$

$R_2 \leftarrow R_2 - 0.1R_1 \quad R_3 \leftarrow R_3 - 0.1R_1$

$$
\left[\begin{array}{ccc|c}
10 & 1 & 1 & 12 \\
0 & 0.9 & 9.9 & 10.8 \\
0 & 9.9 & 0.9 & 10.8
\end{array}\right]
$$

Pivoting

Full Pivoting

$$\max\left(\begin{bmatrix} 0.9 & 9.9 \\ 9.9 & 0.9 \end{bmatrix}\right) = |9.9| = 9.9$$

$$\left[\begin{array}{ccc|c} 10 & 1 & 1 & 12 \\ 0 & 9.9 & 0.9 & 10.8 \\ 0 & 0.9 & 9.9 & 10.8 \end{array}\right]$$

$$\left[\begin{array}{ccc|c} 10 & 1 & 1 & 12 \\ 0 & 9.9 & 0.9 & 10.8 \\ 0 & 0 & 9.818182 & 9.818182 \end{array}\right]$$

Cholesky Factorization

Cholesky Factorization is a specialized matrix decomposition for symmetric and positive definite (SPD) matrices.
It writes a matrix Aas:

$$A = HH^T$$

where
•$A$is symmetric positive definite,
•$H$is a lower triangular matrix with positive diagonal entries.

Symmetry

$$A = A^T$$

Positive definiteness

$$x^T Ax > 0 \quad \text{for all } x \neq 0$$

Cholesky Factorization

Symmetric and positive definite matrices can be written in the form $A = HH^T$ .This factorization is known as Cholesky.

Given $Ax = b$:

1. Compute Cholesky: $A = HH^T$
2. Forward substitution:

$$Hy = b$$

3. Back substitution:

$$H^T x = y$$

Example

$$A = \begin{pmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{pmatrix}, \qquad b = \begin{pmatrix} 4 \\ 6 \\ 7.25 \end{pmatrix}$$

$$A = \begin{bmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{bmatrix} \qquad H = \begin{bmatrix} h_{11} & 0 & 0 \\ h_{21} & h_{22} & 0 \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

$$H^T = \begin{bmatrix} h_{11} & h_{21} & h_{31} \\ 0 & h_{22} & h_{32} \\ 0 & 0 & h_{33} \end{bmatrix}$$

$$HH^T = \begin{bmatrix} h_{11}^2 & h_{11}h_{21} & h_{11}h_{31} \\ h_{21}h_{11} & h_{21}^2 + h_{22}^2 & h_{21}h_{31} + h_{22}h_{32} \\ h_{31}h_{11} & h_{31}h_{21} + h_{32}h_{22} & h_{31}^2 + h_{32}^2 + h_{33}^2 \end{bmatrix}$$

Example

$$A = \begin{pmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{pmatrix}, \qquad b = \begin{pmatrix} 4 \\ 6 \\ 7.25 \end{pmatrix}$$

$$A = \begin{bmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{bmatrix} \qquad H = \begin{bmatrix} h_{11} & 0 & 0 \\ h_{21} & h_{22} & 0 \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

$$H^T = \begin{bmatrix} h_{11} & h_{21} & h_{31} \\ 0 & h_{22} & h_{32} \\ 0 & 0 & h_{33} \end{bmatrix}$$

$$HH^T = \begin{bmatrix} h_{11}^2 & h_{11}h_{21} & h_{11}h_{31} \\ h_{21}h_{11} & h_{21}^2 + h_{22}^2 & h_{21}h_{31} + h_{22}h_{32} \\ h_{31}h_{11} & h_{31}h_{21} + h_{32}h_{22} & h_{31}^2 + h_{32}^2 + h_{33}^2 \end{bmatrix}$$

Example

$$A = \begin{pmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{pmatrix}, \qquad b = \begin{pmatrix} 4 \\ 6 \\ 7.25 \end{pmatrix}$$

Equate $HH^T = A$:

$$\begin{aligned} h_{11}^2 &= 4 \\ h_{11}h_{21} &= -1 \\ h_{11}h_{31} &= 1 \\ h_{21}^2 + h_{22}^2 &= 4.25 \\ h_{21}h_{31} + h_{22}h_{32} &= 2.75 \\ h_{31}^2 + h_{32}^2 + h_{33}^2 &= 3.5 \end{aligned}$$

Example

$$A = \begin{pmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{pmatrix}, \qquad b = \begin{pmatrix} 4 \\ 6 \\ 7.25 \end{pmatrix}$$

$$H = \begin{bmatrix} 2 & 0 & 0 \\ -\frac{1}{2} & 2 & 0 \\ \frac{1}{2} & 1.5 & 1 \end{bmatrix}, \qquad H^T = \begin{bmatrix} 2 & -\frac{1}{2} & \frac{1}{2} \\ 0 & 2 & 1.5 \\ 0 & 0 & 1 \end{bmatrix}$$

$$HH^T = \begin{bmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{bmatrix} = A$$

Example

$$A = \begin{pmatrix} 4 & -1 & 1 \\ -1 & 4.25 & 2.75 \\ 1 & 2.75 & 3.5 \end{pmatrix}, \qquad b = \begin{pmatrix} 4 \\ 6 \\ 7.25 \end{pmatrix}$$

Solve $Hy = b$

$$\begin{pmatrix} 2 & 0 & 0 \\ -0.5 & 2 & 0 \\ 0.5 & 1.5 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 6 \\ 7.25 \end{pmatrix} \qquad y = \begin{pmatrix} 2 \\ 4 \\ 1 \end{pmatrix}$$

Solve $H^T x = y$

$$\begin{pmatrix} 2 & -0.5 & 0.5 \\ 0 & 2 & 1.5 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \\ 1 \end{pmatrix} \qquad x = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

Iterative Solution Methods

- Starts with an initial approximation for the solution vector (x^0)
- At each iteration updates the x vector by using the sytem Ax=b
- During the iterations A, matrix is not changed so sparcity is preserved
- Each iteration involves a matrix-vector product
- If A is sparse this product is efficiently done

Iterative solution procedure

- Write the system Ax=b in an equivalent form

  x=Ex+f (like x=g(x) for fixed-point iteration)
- Starting with x^0, generate a sequence of approximations $\{x^k\}$ iteratively by

  $\mathbf{x^{k+1}=Ex^k+f}$
- Representation of E and f depends on the type of the method used
- But for every method E and f are obtained from A and b, but in a different way

Convergence

- As $k \rightarrow \infty$, the sequence $\{x^k\}$ converges to the solution vector under some conditions on E matrix
- This imposes different conditions on A matrix for different methods
- For the same A matrix, one method may converge while the other may diverge
- Therefore for each method the relation between A and E should be found to decide on the convergence

Different Iterative methods

- Jacobi Iteration
- Gauss-Seidel Iteration
- Successive Over Relaxation (S.O.R)
  - SOR is a method used to accelerate the convergence
  - Gauss-Seidel Iteration is a special case of SOR method

Jacobi iteration

$$
\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\
a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\
&\vdots \\
a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n
\end{aligned}
\qquad
x^0 = \begin{bmatrix} x_1^0 \\ x_2^0 \\ \vdots \\ x_n^0 \end{bmatrix}
$$

$$
x_1^1 = \frac{1}{a_{11}}(b_1 - a_{12}x_2^0 - \cdots - a_{1n}x_n^0)
$$

$$
x_2^1 = \frac{1}{a_{22}}(b_2 - a_{21}x_1^0 - a_{23}x_3^0 - \cdots - a_{2n}x_n^0)
$$

$$
x_n^1 = \frac{1}{a_{nn}}(b_n - a_{n1}x_1^0 - a_{n2}x_2^0 - \cdots - a_{nn-1}x_{n-1}^0)
$$

$$
x_i^{k+1} = \frac{1}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^k - \sum_{j=i+1}^{n} a_{ij}x_j^k \right]
$$

$x^{k+1}=Ex^k+f$ iteration for Jacobi method

A can be written as A=L+D+U (*no decomposition*)

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ a_{21} & 0 & 0 \\ a_{31} & a_{32} & 0 \end{bmatrix} + \begin{bmatrix} a_{11} & 0 & 0 \\ 0 & a_{22} & 0 \\ 0 & 0 & a_{33} \end{bmatrix} + \begin{bmatrix} 0 & a_{12} & a_{13} \\ 0 & 0 & a_{23} \\ 0 & 0 & 0 \end{bmatrix}$$

$Ax=b \Rightarrow (L+D+U)x=b$

$Dx^{k+1} =-(L+U)x^k+b$

$$x_i^{k+1} = \frac{1}{a_{ii}}\left[b_i - \underbrace{\sum_{j=1}^{i-1} a_{ij} x_j^k}_{Lx^k} - \underbrace{\sum_{j=i+1}^{n} a_{ij} x_j^k}_{Ux^k} \right]$$

Dx^{k+1}

$x^{k+1}=-D^{-1}(L+U)x^k+D^{-1}b$

$E=-D^{-1}(L+U)$

$f=D^{-1}b$

Gauss-Seidel (GS) iteration

Use the latest update

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1$$
$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2$$
$$\vdots$$
$$a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n$$

$$x^0 = \begin{bmatrix} x_1^0 \\ x_2^0 \\ \vdots \\ x_n^0 \end{bmatrix}$$

$$x_1^1 = \frac{1}{a_{11}}(b_1 - a_{12}x_2^0 - \cdots - a_{1n}x_n^0)$$
$$x_2^1 = \frac{1}{a_{22}}(b_2 - a_{21}x_1^1 - a_{23}x_3^0 - \cdots - a_{2n}x_n^0)$$
$$x_n^1 = \frac{1}{a_{nn}}(b_n - a_{n1}x_1^1 - a_{n2}x_2^1 - \cdots - a_{nn-1}x_{n-1}^1)$$

$$x_i^{k+1} = \frac{1}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{k+1} - \sum_{j=i+1}^{n} a_{ij}x_j^k \right]$$

$x^{(k+1)}=Ex^{(k)}+f$ iteration for Gauss-Seidel

$Ax=b \Rightarrow (L+D+U)x=b$

$$x_i^{k+1} = \frac{1}{a_{ii}}\left[b_i - \underbrace{\sum_{j=1}^{i-1} a_{ij} x_j^{k+1}}_{Lx^{k+1}} - \underbrace{\sum_{j=i+1}^{n} a_{ij} x_j^{k}}_{Ux^{k}} \right]$$

Dx^{k+1}

$(D+L)x^{k+1} = -Ux^k + b$

$x^{k+1} = -(D+L)^{-1}Ux^k + (D+L)^{-1}b$

$E = -(D+L)^{-1}U$

$f = -(D+L)^{-1}b$

Comparison

- Gauss-Seidel iteration converges more rapidly than the Jacobi iteration since it uses the latest updates
- But there are some cases that *Jacobi* iteration *does converge* but *Gauss-Seidel* does *not*
- To accelerate the Gauss-Seidel method even further, successive over relaxation method can be used

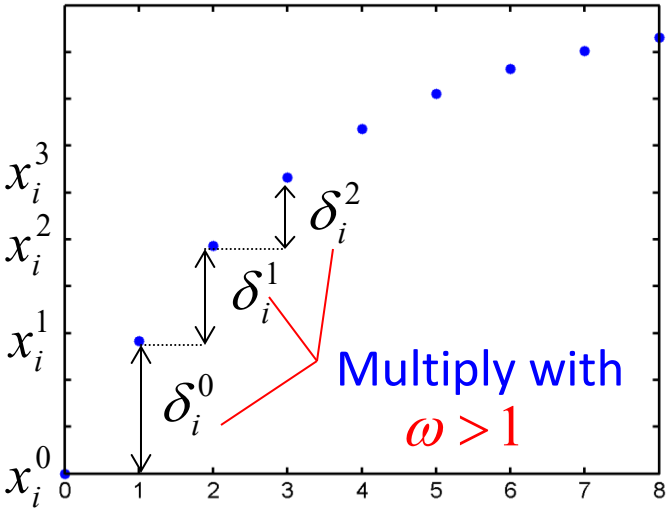
Successive Over Relaxation Method

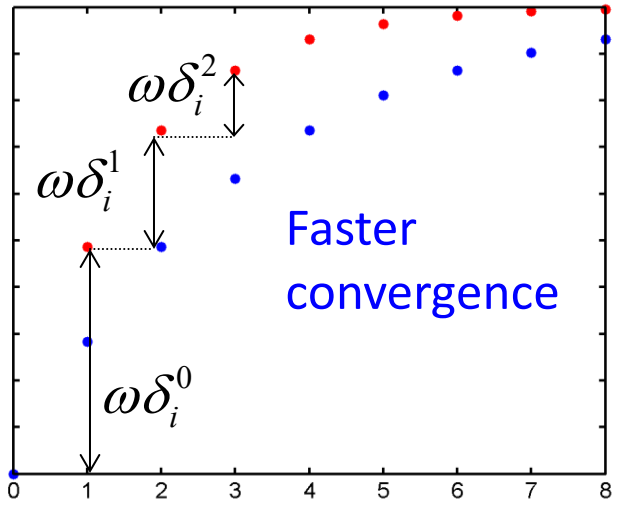
- GS iteration can be also written as follows

$$x_i^{k+1} = x_i^k + \frac{1}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{k+1} - \sum_{j=i}^{n} a_{ij} x_j^k \right]$$

$$x_i^{k+1} = x_i^k + \delta_i^k$$

Correction term





SOR

$$x_i^{k+1} = x_i^k + \omega \delta_i^k$$

$$x_i^{k+1} = x_i^k + \omega \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{k+1} - \sum_{j=i}^{n} a_{ij} x_j^k \right]$$

$$x_i^{k+1} = (1-\omega) x_i^k + \omega \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{k+1} - \sum_{j=i+1}^{n} a_{ij} x_j^k \right]$$

$1<\omega<2$ over relaxation (faster convergence)
$0<\omega<1$ under relaxation (slower convergence)
There is an optimum value for ω
Find it by trial and error (usually around 1.6)

$x^{(k+1)}=Ex^{(k)}+f$ iteration for SOR

$$x_i^{k+1} = (1-\omega)x_i^k + \omega \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{k+1} - \sum_{j=i+1}^{n} a_{ij} x_j^k \right]$$

$Dx^{k+1}=(1-\omega)Dx^k+\omega b-\omega Lx^{k+1}-\omega Ux^k$

$(D+ \omega L)x^{k+1}=[(1-\omega)D-\omega U]x^k+\omega b$

$E=(D+ \omega L)^{-1}[(1-\omega)D-\omega U]$

$f= \omega(D+ \omega L)^{-1}b$

Example 1 (Jacobi Iteration)

$$\begin{bmatrix} 4 & -1 & 1 \\ 4 & -8 & 1 \\ -2 & 1 & 5 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 7 \\ -21 \\ 15 \end{bmatrix} \qquad x^0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \qquad \left\| b - Ax^0 \right\|_2 = 26.7395$$

Diagonally dominant matrix

$$x_1^1 = \frac{7 + x_2^0 - x_3^0}{4} = \frac{7}{4} = 1.75$$

$$x_2^1 = \frac{21 + 4x_1^0 + x_3^0}{8} = \frac{21}{8} = 2.625 \qquad \left\| b - Ax^1 \right\|_2 = 10.0452$$

$$x_3^1 = \frac{15 + 2x_1^0 - x_2^0}{5} = \frac{15}{5} = 3.0$$

Example 1 continued...

$$x_1^2 = \frac{7 + x_2^1 - x_3^1}{4} = \frac{7 + 2.625 - 3}{4} = 1.65625$$

$$x_2^2 = \frac{21 + 4x_1^1 + x_3^1}{8} = \frac{21 + 4 \times 1.75 + 3}{8} = 3.875$$

$$x_3^2 = \frac{15 + 2x_1^1 - x_2^1}{5} = \frac{15 + 2 \times 1.75 - 2.625}{5} = 4.225$$

$$\left\| b - Ax^2 \right\|_2 = 6.7413$$

$$x_1^3 = \frac{7 + 3.875 - 4.225}{4} = 1.6625$$

$$x_2^3 = \frac{21 + 4 \times 1.65625 + 4.225}{8} = 3.98125$$

$$x_3^3 = \frac{15 + 2 \times 1.65625 - 3.875}{5} = 2.8875$$

$$\left\| b - Ax^2 \right\|_2 = 1.9534$$

Matrix is diagonally dominant, Jacobi iterations are *converging*

Example 2

$$\begin{bmatrix} -2 & 1 & 5 \\ 4 & -8 & 1 \\ 4 & -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 15 \\ -21 \\ 7 \end{bmatrix} \qquad x^0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \qquad \left\| b - Ax^0 \right\|_2 = 26.7395$$

The matrix is not diagonally dominant

$$x_1^1 = \frac{-15 + x_2^0 + 5x_3^0}{2} = \frac{-15}{2} = -7.5$$

$$x_2^1 = \frac{21 + 4x_1^0 + x_3^0}{8} = \frac{21}{8} = 2.625$$

$$x_3^1 = 7 - 4x_1^0 + x_2^0 = 7.0$$

$$\left\| b - Ax^1 \right\|_2 = 54.8546$$

Example 2 continued...

$$x_1^1 = \frac{-15 + 2.625 + 5 \times 7}{2} = 11.3125$$

$$x_2^1 = \frac{21 - 4 \times 7.5 + 7}{8} = -0.25$$

$$x_3^1 = 7 + 4 \times 7.5 + 2.625 = 39.625$$

$$\left\| b - Ax^2 \right\|_2 = 208.3761$$

The residual term is increasing at each iteration, so the iterations are *diverging*.

Note that the matrix is not diagonally dominant

Convergence of Gauss-Seidel iteration

- GS iteration converges for any initial vector if A is a *diagonally dominant matrix*
- GS iteration converges for any initial vector if A is a *symmetric* and *positive definite matrix*
- Matrix A is positive definite if

  $x^T Ax > 0$ for every nonzero x vector

Positive Definite Matrices

- A matrix is positive definite if all its eigenvalues are positive
- A symmetric diagonally dominant matrix with positive diagonal entries is positive definite
- If a matrix is positive definite
  - All the diagonal entries are positive
  - The largest (in magnitude) element of the whole matrix must lie on the diagonal

Positive Definitiness Check

$$\begin{bmatrix} 20 & 12 & 25 \\ 12 & 15 & 2 \\ 25 & 2 & 5 \end{bmatrix}$$

Not positive definite
Largest element is not on the diagonal

$$\begin{bmatrix} 20 & 12 & 5 \\ 12 & -15 & 2 \\ 5 & 2 & 25 \end{bmatrix}$$

Not positive definite
All diagonal entries are not positive

$$\begin{bmatrix} 20 & 12 & 5 \\ 12 & 15 & 2 \\ 5 & 2 & 25 \end{bmatrix}$$

Positive definite
Symmetric, diagonally dominant, all diagonal entries are positive

Positive Definitiness Check

$$\begin{bmatrix} 20 & 12 & 5 \\ 12 & 15 & 2 \\ 8 & 2 & 25 \end{bmatrix}$$

A decision can not be made just by investigating the matrix.

The matrix is diagonally dominant and all diagonal entries are positive but it is *not symmetric*.

To decide, check if all the eigenvalues are positive

Example (Gauss-Seidel Iteration)

$$\begin{bmatrix} 4 & -1 & 1 \\ 4 & -8 & 1 \\ -2 & 1 & 5 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 7 \\ -21 \\ 15 \end{bmatrix} \qquad x^0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \qquad \left\| b - Ax^0 \right\|_2 = 26.7395$$

Diagonally dominant matrix

$$x_1^1 = \frac{7 + x_2^0 - x_3^0}{4} = \frac{7}{4} = 1.75$$

$$x_2^1 = \frac{21 + 4x_1^1 + x_3^0}{8} = \frac{21 + 4 \times 1.75}{8} = 3.5$$

$$x_3^1 = \frac{15 + 2x_1^1 - x_2^1}{5} = \frac{15 + 2 \times 1.75 - 3.5}{5} = 3.0$$

$$\left\| b - Ax^1 \right\|_2 = 3.0414$$

$$\left\| b - Ax^1 \right\|_2 = 10.0452$$

Jacobi iteration

Example 1 continued...

$$x_1^2 = \frac{7 + x_2^1 - x_3^1}{4} = \frac{7 + 3.5 - 3}{4} = 1.875$$

$$x_2^2 = \frac{21 + 4x_1^2 + x_3^1}{8} = \frac{21 + 4 \times 1.875 + 3}{8} = 3.9375$$

$$x_3^2 = \frac{15 + 2x_1^2 - x_2^2}{5} = \frac{15 + 2 \times 1.875 - 3.9375}{5} = 2.9625$$

$$\left\| b - Ax^2 \right\|_2 = 0.4765$$

$$\left\| b - Ax^2 \right\|_2 = 6.7413$$

Jacobi iteration

When both Jacobi and Gauss-Seidel iterations converge, Gauss-Seidel converges faster

Convergence of SOR method

- If $0<\omega<2$, SOR method converges for any initial vector if A matrix is symmetric and positive definite
- If $\omega>2$, SOR method diverges
- If $0<\omega<1$, SOR method converges but the convergence rate is slower (deceleration) than the Gauss-Seidel method.

Operation count

- The operation count for Gaussian Elimination or LU Decomposition was $O(n^3)$, order of n^3.
- For iterative methods, the number of scalar multiplications is $O(n^2)$ at each iteration.
- If the total number of iterations required for convergence is much less than n, then iterative methods are more efficient than direct methods.
- Also iterative methods are well suited for sparse matrices